La théorie des nombres

Error message

```
UndefVarError: `bibcite` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
md"""
* Théorie des nombres: $(cite("hoffstein2014Introduction", "1.2, 1.3, 1.4, 1.5, 2.2, 2.3"))
* Discrete Logarithme Problem et Diffie-Hellman: $(cite("hoffstein2014Introduction", "2.2, 2.3, 2.6, 2.7, 2.8"))
"""
```

👀 Reading hidden code

Error message

```
UndefVarError: `bibrefs` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
refs(["hoffstein2014Introduction"])
```

👀 Reading hidden code

Error message

```
UndefVarError: `frametitle` not defined in `Main.var"workspace#3"`
```

```
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
frametitle("Exemples")
```

Reading hidden

Si tous les mois avaient 30 jours, est-ce qu'il y a des jours de la semaine qui ne seront jamais le premier du mois ?

Reformulation: pour tout nombre $0 \leq j < 7$, existe-t-il x et y tels que $30x = j + 7y$. Notation modulo : $30x \equiv j \pmod{7}$.

Si tous les ans avaient 365 jours, est-ce qu'il y a des jours de la semaine qui ne seront jamais le 25 Décembre ? Est si tous les ans avaient 366 jours ? Et s'ils avaient 364 jours ?

Reformulation: pour tout nombre $0 \leq j < 7$, existe-t-il x et y tels que $365x = j + 7y$. Notation modulo : $365x \equiv j \pmod{7}$.

Error message

```
UndefVarError: `frametitle` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
frametitle("Théorème de Bézout")
```

Reading hidden

Définition Le *Greatest Common Divisor (GCD)* de deux nombres $a \in \mathbb{Z}$ et $b \in \mathbb{Z}$, noté $\gcd(a, b)$ est le plus grand nombre $g \in \mathbb{Z}$ qui divise a (noté $g \mid a$) et b (noté $g \mid b$). C'est à dire qu'il existe $x \in \mathbb{Z}$ tel que $a = gx$ et $y \in \mathbb{Z}$ tel que $b = gy$. En notation modulaire, $a \equiv 0 \pmod{g}$ et $b \equiv 0 \pmod{g}$.

Théorème de Bézout Il existe $x, y \in \mathbb{Z}$ tels que $ax + by = c$ si et seulement si $\gcd(a, b)$

divise c. En notation modulaire $ax \equiv c \pmod{b}$ et $by \equiv c \pmod{a}$.

Error message

```
UndefVarError: `qa` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
qa(md"Comment prouver que l'égalité ``ax + by = c`` implique que ``\text{gcd}(a, b)`` divise ``c`` ?",
md"""
Soit ``g = \text{gcd}(a, b)``. Par définition, il existe ``\alpha, \beta`` tels que ``a = \alpha g`` et ``b = \beta g``.
On a alors ``c = ax + by = (\alpha x + \beta y) g`` ce qui implique que ``c`` est un multiple de ``g``.
""",)
```

Reading hidden

Error message

```
UndefVarError: `frametitle` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
frametitle("Algorithme d'Euclide : élaboration")
```

Reading hidden

Définition Le résultat de la *division Euclidienne* de a par un diviseur d est un quotient q et un reste $0 \leq r < d$ tels que $a = qd + r$. En notation modulaire $a \equiv r \pmod{d}$.

Error message

```
UndefVarError: `qa` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
qa(md"**Observation clé** Que dit le théorème de Bézout par rapport à ``\text{gcd}(a, d)`` et ``r``.",
md"""
Le reste ``r`` est **divisible** par ``\text{gcd}(a, d)``.
Le nombre ``\text{gcd}(a, d)`` divise donc les 3 nombres, ``a``, ``d`` et ``r`` et donc ``\text{gcd}(a, d) = \text{gcd}(a, d, r)``.
""")
```

Reading hidden

Error message

```
UndefVarError: `qa` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
qa(md"**Observation clé** Que dit le théorème de Bézout par rapport à ``\text{gcd}(d, r)`` et ``a``.",
md"""
Le nombre ``a`` est **divisible** par ``\text{gcd}(d, r)``.
Le nombre ``\text{gcd}(d, r)`` divise donc les 3 nombres, ``a``, ``d`` et ``r`` et donc ``\text{gcd}(d, r) = \text{gcd}(a, d, r)``.
En combinant ça avec l'observation précédente, on a ``\text{gcd}(a, d) = \text{gcd}(d, r)``. On peut généraliser cela en le lemme suivant:
""")
```

Reading hidden

Lemme: Si $a \equiv r \pmod{b}$ alors $\gcd(a, b) = \gcd(b, r)$.

Error message

```
UndefVarError: `qa` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
qa(md"**Observation clé** Si ``a > b``, trouver un mono-variant.",
md"""
On a ``(a, b) > (b, r)``. En effet, ``a > b`` par supposition et ``b > r`` par définition de l'algorithme d'Euclide.
Notons que même si la supposition ``a > b`` n'est pas vraie, elle le devient pour ``\text{gcd}(b, r)``.
""")
```

👀 Reading hidden code

Error message

```
UndefVarError: `qa` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
qa(md"**Observation finale** Si ``a`` et ``b`` sont positifs et qu'on effectue la substitution ``(a, b) \to (b, r)`` récursivement, le mono-variant impose qu'on ne puisse itérer qu'un nombre fini de fois, que va-t-il se passer ?",
md"""
La paire ``(a, b)`` va diminuer strictement (c'est à dire d'au moins 1) à chaque itération. Pourtant, ce sont des nombres entier positifs donc ils ne peuvent diminuer strictement qu'un nombre fini de fois. C'est une contradiction, comme cela se fait-il ?
À un moment ``b`` vaudra 0, on ne pourra alors plus faire de division Euclidienne. On utilisera alors le fait que ``\text{gcd}(a, 0) = a``.
""")
```

👀 Reading hidden code

Error message

```
UndefVarError: `frametitle` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```julia
frametitle("Algorithme d'Euclide : implémentation")
```

Reading hidden

pgcd (generic function with 1 method)

```julia
function pgcd(a, b)
    println("gcd($a, $b) = ")
    if b == 0
        println(a)
        return a
    else
        return pgcd(b, mod(a, b))
    end
end
```

gcd_a = 90284599

gcd_b = 249357461

1

```julia
pgcd(gcd_a, gcd_b)
```

```
gcd(90284599, 249357461) =
gcd(249357461, 90284599) =
gcd(90284599, 68788263) =
gcd(68788263, 21496336) =
gcd(21496336, 4299255) =
gcd(4299255, 61) =
gcd(61, 36) =
gcd(36, 25) =
gcd(25, 11) =
gcd(11, 3) =
gcd(3, 2) =
gcd(2, 1) =
gcd(1, 0) =
1
```

The complexity is difficult to evaluate but can be shown to be $O(\log(\min(a, b)))$.

Error message

```
UndefVarError: `frametitle` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
frametitle("Arithmétique modulaire : somme")
```

👀 Reading hidden

$$a \equiv \alpha \pmod{n} \quad \text{et} \quad b \equiv \beta \pmod{n} \quad \Rightarrow \quad a + b \equiv \alpha + \beta \pmod{n}$$

a	α	b	β	n
11	1	13	3	5

```
abn_picker
```

4

```
mod(a + b, n)
```

4

```
mod(mod(a, n) + mod(b, n), n)
```

Error message

```
UndefVarError: `frametitle` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
frametitle("Arithmétique modulaire : produit")
```

👀 Reading hidden

$$a \equiv \alpha \pmod{n} \quad \text{et} \quad b \equiv \beta \pmod{n} \quad \Rightarrow \quad ab \equiv \alpha\beta \pmod{n}$$

a	α	b	β	n
11	1	13	3	5

3

```
mod(a * b, n)
```

3

```
mod(mod(a, n) * mod(b, n), n)
```

Corollaire

$$n \mid a \quad \text{et} \quad n \mid b \quad \Rightarrow \quad n \mid (ab)$$

À ne pas confondre avec

$$a \mid n \quad \text{et} \quad b \mid n \quad \Rightarrow \quad (ab/\gcd(a,b)) \mid n$$

Error message

```
UndefVarError: `frametitle` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
frametitle("Division par 3 et 9")
```

Reading hidden

Est-ce que 2345 est divisible par 3 ou 9?

$$\begin{aligned} 2 \cdot 10^3 + 3 \cdot 10^2 + 4 \cdot 10 + 5 &\equiv ? \pmod{9} \\ 2 \cdot 1^3 + 3 \cdot 1^2 + 4 \cdot 1 + 5 &\equiv ? \pmod{9} \\ 2 + 3 + 4 + 5 &\equiv 14 \pmod{9} \end{aligned}$$

Est-ce que 2345 est divisible par 11?

$$\begin{aligned} 2 \cdot (10)^3 + 3 \cdot 10^2 + 4 \cdot 10 + 5 &\equiv ? \pmod{11} \\ 2 \cdot (-1)^3 + 3 \cdot (-1)^2 + 4 \cdot (-1) + 5 &\equiv ? \pmod{11} \\ -2 + 3 - 4 + 5 &\equiv 2 \pmod{11} \end{aligned}$$

Error message

```
UndefVarError: `frametitle` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
frametitle("Inverse et division modulaire")
```

Reading hidden

- **Inverse modulaire** : étant donné a, n, trouver x (noté a^{-1}) tel que $xa \equiv 1 \pmod{n}$
- **Division modulaire** : étant donné a, b, n, trouver x tel que $xa \equiv b \pmod{n}$ → $x \equiv a^{-1}b \pmod{n}$.

Error message

```
UndefVarError: `qa` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
qa(md"Est-ce que l'inverse modulaire existe toujours ?",
md"""
Par le théorème de Bézout, il existe si et seulement si ``\text{gcd}(a, n) \mid 1``, c'est à dire que ``\text{gcd}(a, n) = 1``.
""")
```

Reading hidden

Error message

```
UndefVarError: `qa` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
qa(md"Comment trouver l'inverse modulaire ?",
md"""
Si on avait les coefficients ``x`` et ``y`` tels que ``xa + yn = 1``, l'inverse serait ``x``. Dans l'algorithme d'Euclide, on ne garde que le **reste** et on oublie le **quotient**. Il faudrait combiner les quotients des différentes opérations pour trouver ``x``.
""")
```

Reading hidden code

Error message

```
UndefVarError: `frametitle` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
frametitle("Algorithme d'Euclide étendu")
```

Reading hidden code

$$xb + yr = g \quad \text{et} \quad r = a - qb \quad \Rightarrow \quad (x - yq)b + ya = g$$

Solution *homogène* $x = b, y = -a \to ba - ab = 0$. Donc si (x, y) est solution, $(x + b, y - a)$ aussi.

```
pgcdx (generic function with 1 method)
```

```
function pgcdx(a, b)
    if b == 0
        return a, one(a), zero(a)
    else
        q, r = divrem(a, b)
        g, x, y = pgcdx(b, r)
        return g, y, x - y * q
    end
end
```

```
(1, 89932200, -32561659)
```

```
gcd_g, gcd_x, gcd_y = pgcdx(gcd_a, gcd_b)
```

```
1
```

```julia
gcd_x * gcd_a + gcd_y * gcd_b
```

Pas une solution unique:

```
(1, 89932200, -32561659)
```

```julia
gcdx(gcd_a, gcd_b)
```

Error message

```
UndefVarError: `frametitle` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```julia
frametitle("Inversion modulaire par Euclide étendu")
```

Reading hidden

Ensemble de solutions: $(x + kb, y - ka)$ pour un $k \in \mathbb{Z}$ arbitraire. Prenons k tel que $0 \leq x + kb < b$ avec `mod`.

```
modinv (generic function with 1 method)
```

```julia
function modinv(a, n)
    g, x, y = gcdx(a, n)
    return mod(x, n)
end
```

Revenons aux exemples:

$$30x \equiv j \pmod{7} \quad \Rightarrow \quad x \equiv (30)^{-1} j \pmod{7}$$

```
[0, 4, 1, 5, 2, 6, 3]
```

```julia
collect(mod.(modinv(30, 7) .* (0:6), 7))
```

$$365x \equiv j \pmod{7} \quad \Rightarrow \quad x \equiv (365)^{-1} j \pmod{7}$$

```
[0, 1, 2, 3, 4, 5, 6]
```

```julia
collect(mod.(modinv(365, 7) .* (0:6), 7))
```

$$366x \equiv j \pmod 7 \quad \Rightarrow \quad x \equiv (366)^{-1} j \pmod 7$$

```
[0, 4, 1, 5, 2, 6, 3]
```

```julia
collect(mod.(modinv(366, 7) .* (0:6), 7))
```

S'il y avait 364 jours par ans, les fêtes seraient toujours le même jour de la semaine!

```
7
```

```julia
gcd(364, 7)
```

Error message

```
UndefVarError: `frametitle` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```julia
frametitle("Fast powering")
```

Reading hidden

Comment calculer a^m pour un large m ?

power = 251

```
361850278866613110698659328152149712041468702080126762623304950024728530124 8
```

```julia
@time big(2)^power
```

```
0.000013 seconds (5 allocations: 160 bytes)
```

Error message

```
UndefVarError: `qa` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
qa(md"Supposons que ``m`` est pair, c'est à dire ``m = 2k``...",
md"""
On a ``a^(2k) = (a^k)^2``. Si ``b = a^k``, on calcule le produit ``b \times b``.
""")
```

Reading hidden

Error message

```
UndefVarError: `qa` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
qa(md"Que faire si que ``m`` est impair, c'est à dire ``m = 2k + 1``...",
md"""
Si ``b = a^(2k)``, calcule le produit ``b \times a``.
""")
```

Reading hidden

Error message

```
UndefVarError: `frametitle` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
frametitle("Recursive implementation")
```

Reading hidden

```
fast_power (generic function with 1 method)
```

```
function fast_power(prod_func::Function, a, power)
    if power == 0
        return one(a)
    elseif mod(power, 2) == 1
        return prod_func(fast_power(prod_func, a, power - 1), a)
    else
        b = fast_power(prod_func, a, div(power, 2))
        return prod_func(b, b)
    end
end
```

power = 251

```
361850278866613110698659328152149712041468702080126762623304950024728530124 8
```

```
@time big(2)^power
```

```
  0.000006 seconds (5 allocations: 160 bytes)
```

```
361850278866613110698659328152149712041468702080126762623304950024728530124 8
```

```
@time fast_power(*, big(2), power)
```

```
  0.008504 seconds (3.39 k allocations: 176.500 KiB, 99.65% compilation tim
e)
```

Error message

```
UndefVarError: `qa` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
qa(md"Quelle est la complexité temporelle ?", md"Si ``n`` est impair, au coup
suivant, il est pair donc il n'est impair qu'au pire une fois sur deux. En ``l``
multiplication, on divise ``m`` au moins par ``2^{l/2}`` donc on a une complexité
logarithmique ``\Theta(\log(m))``  en supposant que `prod_func` a une complexité
``\Theta(1)``.")
```

👀 Reading hidden

Error message

```
UndefVarError: `frametitle` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
frametitle("Fast modular powering")
```

👀 Reading hidden

```
fast_mod_power (generic function with 1 method)
```

Last 3 digit:

```
@time pow_1000 = 248
```

```
@time pow_1000 = fast_mod_power(2, power, 1000)
```

```
0.000001 seconds
```

Et modulo 999 ?

```
@time pow_999 = 500
```

```
@time pow_999 = fast_mod_power(2, power, 999)
```

```
0.000001 seconds
```

Par l'algo d'Euclide, $\gcd(n, n-1) = 1$ donc $\gcd(1000, 999) = 1$.

Error message

```
UndefVarError: `qa` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
qa(md"Comment trouver `mod(2^power, 999000)` en utilisant `pow_1000` et `pow_999`
?", md"
On peut utiliser une astuce similaire à l'interpolation Lagrangienne.
On veut trouver ``x`` et ``y`` tels que
```math
\texttt{pow_1000}x + \texttt{pow_999}y \equiv 2^\texttt{power} \pmod{999000}
```
On veut que ``x \equiv 0 \pmod{999}`` et ``x \equiv 1 \pmod{1000}``.
On utilise donc ``x = 999x'`` avec ``x' \equiv (999)^{-1} \pmod{1000}``.
")
```

Reading hidden

252248

```
fast_mod_power(2, power, 999000)
```

252248

```
mod(pow_1000 * 999 * modinv(999, 1000) + pow_999 * 1000 * modinv(1000, 999), 999000)
```

Error message

```
UndefVarError: `frametitle` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
frametitle("Chinese remainder theorem")
```

Reading hidden

Error message

```
UndefVarError: `bibcite` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
md"""
Voir $(cite("hoffstein2014Introduction", "Section 2.8")).
"""
```

Reading hidden

chinese_remainder_theorem (generic function with 1 method)

252248

```
chinese_remainder_theorem([pow_1000, pow_999], [1000, 999])
```

primes_upper = 100

prime_list =
[3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]

```
prime_list = primes(3, primes_upper)
```

```
[2, 3, 4, 2, 7, 8, 10, 6, 15, 2, 19, 39, 22, 12, 50, 14, 35, 41, 64, 37, 11, 32, 67, 11]
```

```
fast_mod_power.(2, power, prime_list)
```

```
986325341584402112586461440731025613
```

```
chinese_remainder_theorem(big.(fast_mod_power.(2, power, prime_list)), big.(prime_list))
```

```
36185027886661311069865932815214971204146870208012676262330495002472853012 48
```

```
big(2)^power
```

Error message

```
UndefVarError: `qa` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
qa(md"Comment savoir si `prime_list` contient assez de nombres pour avoir la bonne réponse ?", md"On a la bonne réponse modulo `prod(prime_list)` donc si `prod(prime_list) > 2^power`, on a la bonne réponse.")
```

Reading hidden

Error message

```
UndefVarError: `frametitle` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
frametitle("Fibonacci sequence")
```

Reading hidden

Error message

```
UndefVarError: `HAlign` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
HAlign(
md"""
Équation de récurrence:
```math
x_{k+1} = x_k + x_{k-1}
```
Reformulation sans ``(k-1)``
```math
\begin{align}
x_{k+1} & = x_k + y_{k}\\
y_{k+1} & = x_k
\end{align}
```
Forme matricielle:
```math
\begin{bmatrix}
x_{k+1}\\
y_{k+1}
\end{bmatrix}
=
\begin{bmatrix}
1 & 1\\
1 & 0
\end{bmatrix}
\begin{bmatrix}
x_{k}\\
y_{k}
\end{bmatrix}
```
Matrix power:
```math
\begin{bmatrix}
x_n\\
y_n
\end{bmatrix}
=
\begin{bmatrix}
1 & 1\\
1 & 0
\end{bmatrix}^n
\begin{bmatrix}
```
```

```
x_0\\
y_0
\end{bmatrix}
```
""",
md"""
`n` = $(fib_picker)\

$(draw_fib(fib_n))
""",
)

Reading hidden

Error message

```
UndefVarError: `frametitle` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
frametitle("Fast powering for matrices")
```

Reading hidden

fib_rec (generic function with 1 method)

```
fib_rec(n) = (n == 0 ? 0 : (n == 1 ? 1 : fib_rec(n - 1) + fib_rec(n - 2)))
```

267914296

```
@time fib_rec(42)
```

```
1.647984 seconds
```

fib_seq (generic function with 1 method)

```
function fib_seq(n)
    f = zeros(BigInt, n + 1)
    f[2] = 1
    for k in 2:n
        f[k + 1] = f[k] + f[k - 1]
    end
    return f[end]
end
```

```
25311623237323612422401550035206072917663564858024852789519298419913127817605413152301534
```

```julia
@time fib_seq(20000)
```

```
0.020316 seconds (40.01 k allocations: 17.619 MiB, 79.88% gc time)
```

```
fib_pow (generic function with 1 method)
```

```julia
function fib_pow(n)
    A = BigInt[1 1
         1 0]
    x = A^(n-1) * [1, 0]
    return x[1]
end
```

```
25311623237323612422401550035206072917663564858024852789519298419913127817605413152301534
```

```julia
@time fib_pow(20000)
```

```
0.000545 seconds (1.85 k allocations: 201.812 KiB)
```

Error message

```
UndefVarError: `frametitle` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```julia
frametitle("Diagonalization to speed up powering")
```

Reading hidden

```
E = Eigen{Float64, Float64, Matrix{Float64}, Vector{Float64}}
    values:
    2-element Vector{Float64}:
     -0.6180339887498948
      1.618033988749895
    vectors:
    2×2 Matrix{Float64}:
      0.525731  -0.850651
     -0.850651  -0.525731
```

```julia
E = eigen([1 1; 1 0])
```

```
D = 2×2 Diagonal{BigFloat, Vector{BigFloat}}:
     -0.618034    ⋅
        ⋅        1.61803
```

```julia
D = Diagonal([(1 - √big(5)) / 2, (1 + √big(5)) / 2])
```

```
2×2 Matrix{Float64}:
  0.525731  -0.850651
 -0.850651  -0.525731
```

```julia
E.vectors
```

```
2×2 Matrix{Float64}:
 1.0   1.0
 1.0  -1.11022e-16
```

```julia
E.vectors * Diagonal(E.values) * inv(E.vectors)
```

```
fib_diag (generic function with 1 method)
```

```julia
function fib_diag(n)
    x = E.vectors * D^(n - 1) * (E.vectors \ [1, 0])
    return x[1]
end
```

```
2.531162323732361578998428490601662084769270923897910687031954402219719762339543e+4179
```

```julia
@time fib_diag(20000)
```

```
  1.464572 seconds (2.95 M allocations: 148.370 MiB, 12.20% gc time, 99.99% compilation time)
```

```
25311623237323612422401550035206072917663564858024852789519298419913127817605413152301534
```

```julia
@time fib_pow(20000)
```

```
  0.000303 seconds (1.85 k allocations: 201.953 KiB)
```

Error message

```
UndefVarError: `frametitle` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```julia
frametitle("Closed form solution")
```

Trouver b tel que x_k est solution:

$$x_k = b^k \quad \rightarrow \quad b^{k+1} = b^k + b^{k-1} \quad \rightarrow \quad b^2 - b - 1 = 0 \quad \rightarrow \quad b = \frac{1 \pm \sqrt{5}}{2}$$

On a donc une famille de solutions:

$$x_k = a_1 \left(\frac{1 - \sqrt{5}}{2} \right)^k + a_2 \left(\frac{1 + \sqrt{5}}{2} \right)^k$$

Il reste à trouver a_1 et a_2 tels que $x_0 = 0$ et $x_1 = 1$. Ça correspond à calculer `E.vectors \ [1, 0]`, etc...

$$\begin{aligned} x_0 &= 0 & a_1 + a_2 &= 0 \\ x_1 &= 1 & a_1 \frac{1 - \sqrt{5}}{2} + a_2 \frac{1 + \sqrt{5}}{2} &= 1 \end{aligned}$$

Donc $a_1 = -1/\sqrt{5}$ et $a_2 = 1/\sqrt{5}$.

```
fib_closed (generic function with 1 method)
```

```
fib_closed(n) = (((1 + √big(5)) / 2)^n - ((1 - √big(5)) / 2)^n) / √big(5)
```

```
2.531162323732361242240155003520607291766356485802485278951929841991312781989138e+4179
```

```
@time fib_closed(20000)
```

```
0.000031 seconds (36 allocations: 1.695 KiB)
```

```
2.531162323732361578998428490601662084769270923897910687031954402219719762339543e+4179
```

```
@time fib_diag(20000)
```

```
0.000044 seconds (58 allocations: 2.750 KiB)
```

```
25311623237323612422401550035206072917663564858024852789519298419913127817605413152301534
```

```
@time fib_pow(20000)
```

```
0.000281 seconds (1.85 k allocations: 201.969 KiB)
```

Error message

```
UndefVarError: `frametitle` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
frametitle("Fermat's Little Theorem")
```

Reading hidden code

Error message

```
UndefVarError: `bibcite` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
md"""
**Fermat's little theorem** $(cite("hoffstein2014Introduction", "Theorem 1.24"))

```math
\text{Si} \quad p \text{ est premier}\quad \text{et} \quad p \nmid g,\quad \text{alors} \quad
g^{p - 1} \equiv 1 \pmod{p}.
```
"""
```

Reading hidden code

Définition g est une *racine primitive* modulo p si g^k prend toutes les valeurs $1, 2, \ldots, p-1$.

$$\text{Si} \quad p \nmid b, \quad \text{alors} \quad b^{p-1} \equiv 1 \pmod{p}$$

Error message

Another cell defining gp_picker contains errors.

```
gp_picker
```

Reading hidden code

Error message

> *Another cell defining g contains errors.*

```
all_powers = fast_mod_power.(g, 1:(p-1), p)
```

Error message

> *Another cell defining g contains errors.*

```
sort(all_powers)
```

Error message

> *Another cell defining g contains errors.*

```
unique(sort(all_powers))
```

Error message

> *Another cell defining g and all_powers contains errors.*

```
if length(unique(sort(all_powers))) == p - 1
    md"Le nombre $g **est** une racine primitive modulo $p"
else
    md"Le nombre $g **n'est pas** une racine primitive modulo $p"
end
```

👀 *Reading hidden*

Error message

```
UndefVarError: `frametitle` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
frametitle("Dicrete logarithm")
```

Reading hidden

Étant donné un nombre premier p et une racine primitive g modulo p et un entier a tel que $p \nmid a$, le *Discrete logarithme problem* consiste à retrouver x tel que $g^x \equiv a \pmod{p}$.

```
discrete_log (generic function with 1 method)
```

```
function discrete_log(a, g, p)
    gx = one(a)
    for x = 0:(p-2)
        if a == gx
            return x
        end
        gx = mod(gx * g, p)
    end
end
```

Error message

Another cell defining gp_picker contains errors.

```
gp_picker
```

Reading hidden

Error message

Another cell defining g and all_powers contains errors.

```
if length(unique(sort(all_powers))) == p - 1
    md"Le nombre $g **est** une racine primitive modulo $p"
else
    md"Le nombre $g **n'est pas** une racine primitive modulo $p"
end
```

Reading hidden

Error message

Another cell defining g contains errors.

```
x = discrete_log(3, g, p)
```

Error message

Another cell defining g and x contains errors.

```
fast_mod_power(g, x, p)
```

Error message

```
UndefVarError: `qa` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
qa(md"Quelle est la complexité spatiale et temporelle de `discrete_log` ?", md"""
``\mathcal{O}(p)`` temporelle et ``\Omega(1)`` spatiale. Voir
$(cite("hoffstein2014Introduction", "Proposition 2.19")).
""")
```

👀 Reading hidden code

Error message

```
UndefVarError: `qa` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
qa(md"Est-ce une complexité linéaire ou exponentielle en fonction de la taille de l'input",
md"""
La taille est proportionnelle à ``\log_2(p)`` donc on être linéaire en ``p`` c'est être proportionnel à ``2^{\log_2(p)}`` et donc la complexité est exponentielle en la taille de l'input ! $(cite("hoffstein2014Introduction", "Section 2.6"))
""")
```

Reading hidden

Error message

```
UndefVarError: `frametitle` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
frametitle("Meet in the middle approach")
```

Reading hidden

La méthode *meet in the middle* est une méthode générique permettant de passer d'une complexité de $\mathcal{O}(N)$ à $\mathcal{O}(\sqrt{N})$.

Error message

```
Another cell defining g contains errors.
```

```
fast_mod_power.(g, 0:15, 17)
```

On peut mettre le vecteur de taille n^2 sous forme de matrice de taille $n \times n$

Error message

> *Another cell defining g contains errors.*

```
reshape(fast_mod_power.(g, 0:15, 17), 4, 4)
```

On remarque que la matrice est de rang 1. Elle vaut

$$\begin{bmatrix} 1 & g^n & \cdots & g^{n^2-n} \\ g & g^{n+1} & \ddots & g^{n^2-n+1} \\ \vdots & \ddots & \ddots & \vdots \\ g^{n-1} & g^{2n-1} & \cdots & g^{n^2-1} \end{bmatrix} \equiv \begin{bmatrix} 1 \\ g \\ g^2 \\ \vdots \\ g^{n-1} \end{bmatrix} \begin{bmatrix} 1 & g^n & g^{2n} & \cdots & g^{n^2-n} \end{bmatrix} \pmod{p}$$

Error message

> *Another cell defining g contains errors.*

```
mod.(fast_mod_power.(g, 0:3, 17) * fast_mod_power.(g^4, 0:3, 17)', 17)
```

On doit donc trouver la ligne i et la ligne et j tels que

$$\begin{aligned} g^{i-1}g^{(j-1)n} &\equiv a && \pmod{p} \\ g^{i-1} &\equiv a(g^{-n})^{j-1} && \pmod{p} \end{aligned}$$

Ils ne reste plus qu'à chercher une collision entre les listes de restes modulo p pour g^{i-1} et $a(g^{-n})^{j-1}$. L'identification des collision peut se faire en $\mathcal{O}(\sqrt{n}\log(n))$ avec une recherche dichotomique our en $\mathcal{O}(\sqrt{n})$ amorti avec un dictionaire.

Error message

```
UndefVarError: `frametitle` not defined in `Main.var"workspace#3"`
```

```
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
frametitle("Shanks's Babystep-Giantstep Algorithm")
```

Reading hidden

Error message

```
UndefVarError: `bibcite` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
md"""Voir $(cite("hoffstein2014Introduction", "Section 2.7"))"""
```

Reading hidden

giant_steps (generic function with 1 method)

```
function giant_steps(g, n, p)
    gn = fast_mod_power(g, n, p)
    return baby_steps.(modinv(gn, p), n, p)
end
```

baby_steps (generic function with 1 method)

```
function baby_steps(g, n, p)
    steps = [one(g)]
    for i in 1:n
        push!(steps, mod(steps[end] * g, p))
    end
    return steps
end
```

collision (generic function with 1 method)

```
function collision(a, b)
    d = Dict(a[i] => i for i in eachindex(a))
    for j in eachindex(b)
        if haskey(d, b[j])
            return d[b[j]], j
        end
    end
end
```

shanks_discrete_log (generic function with 1 method)

```
function shanks_discrete_log(a, g, p)
    n = isqrt(p) + 1
    i, j = collision(baby_steps(g, n, p), mod.(a .* giant_steps(g, n, p), p))
    return i - 1 + (j - 1) * n
end
```

Error message

> *Another cell defining g contains errors.*

```
shanks_x = shanks_discrete_log(3, g, p)
```

Error message

> *Another cell defining g and shanks_x contains errors.*

```
fast_mod_power(g, shanks_x, p)
```

Error message

```
UndefVarError: `qa` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
qa(md"Quelle est la complexité?", md"``\mathcal{O}(\sqrt{p}\log(p))``")
```

Reading hidden

Error message

```
UndefVarError: `frametitle` not defined in `Main.var"workspace#3"`
```

```
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
frametitle("Diffie-Hellman")
```

Reading hidden

Étant donné un nombre premier p et une racine primitive g modulo p, Alice (resp. Bob) génère un nombre secret a (resp. b). Ils communique ensuite publiquement A et B.

$$A \equiv g^a \pmod{p} \qquad B \equiv g^b \pmod{p}$$

$$A' \equiv B^a \pmod{p} \qquad B' \equiv A^b \pmod{p}$$

Error message

```
UndefVarError: `qa` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
qa(md"What is the relation between ``A'`` and ``B'`` ?",
md"""
```math
A' \equiv (g^b)^a \equiv g^{ab} \equiv (g^{a})^b \equiv B' \pmod{p}
```
Alice et Bob ont donc maintenant la même clef! Il est cependant difficile de trouver ``A'`` depuis ``A`` et ``B`` sans connaitre les secrets ``a`` ou ``b`` si le Discrete Logarithm Problem est difficile.
""")
```

Reading hidden

Error message

```
UndefVarError: `bibcite` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
md"""Voir $(cite("hoffstein2014Introduction", "Section 2.3"))"""
```

Reading hidden

Error message

```
UndefVarError: `section` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
section("Utils")
```

Reading hidden

[2, 3, 5, 7]

```
Primes.primes(10)
```

```
using PlutoUI, Primes, DataFrames, Luxor, Colors, LinearAlgebra
```

```
import DocumenterCitations
```

slider_a = 11

```
slider_a = @bind a Slider(1:100, default=11, show_value = true)
```

slider_b = 13

```
slider_b = @bind b Slider(1:100, default=13, show_value = true)
```

slider_n = 5

```
slider_n = @bind n Slider(1:100, default=5, show_value = true)
```

```
abn_picker =
```

a	α	b	β	n
11	1	13	3	5

Error message

```
UndefVarError: `HAlign` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
gp_picker = HAlign(
    md"`g` = $(@bind g Slider(2:(p-1), default = 2, show_value = true))",
    md"`p` = $p_picker",
)
```

power_slider = 251

```
power_slider = @bind power Slider(1:10000, default = 256, show_value = true)
```

draw_fib (generic function with 2 methods)

```julia
function draw_fib(n, size = 400)
    f = [0, 1, 1]
    for i in 3:(n+1)
        push!(f, f[end] + f[end - 1])
    end
    scale = div(size, 2maximum(f[end-1]))
    #Luxor.scale(scale)
    colors = distinguishable_colors(n)
    if iseven(n)
        Δx = f[end]
        Δy = f[end - 1]
    else
        Δx = f[end - 1]
        Δy = f[end]
    end
    left_most = sum(f[i] for i in 1:(n+1) if mod(i, 4) == 1; init = 0)
    up_most = sum(f[i] for i in 1:(n+1) if mod(i, 4) == 0; init = 0)
    shift = Point(left_most - Δx / 2, up_most - Δy / 2)
    pos(x, y) = scale * (Point(x, y) + shift)
    @draw begin
        x = 0
        y = 0
        j = 1
        for i in 2:(n+1)
            left = x
            if isodd(i)
                if iseven(div(i - 1, 2))
                    left -= f[i]
                else
                    left += f[i - 1]
                end
            end
            up = y
            if iseven(i)
                if iseven(div(i, 2))
                    up -= f[i]
                else
                    up += f[i-1]
                end
            end
            sethue(colors[i - 1])
            setopacity(0.6)
            rect(pos(left, up), scale * f[i], scale * f[i], action=:fill)
            setopacity(1)
            sethue("black")
            fontsize(div(scale * f[i], 2))
            text(string(f[i]), pos(left + f[i] / 2, up + f[i] / 2), halign =
            :center, valign = :middle)
```

```
            x = min(x, left)
            y = min(y, up)
        end
    end Δx * scale Δy * scale
```

fib_picker = 10

```
fib_picker = @bind fib_n Slider(1:12, default = 10, show_value = true)
```

p_picker = 11

```
p_picker = @bind p Slider(primes(20), default = 11, show_value = true)
```

Error message from Main

```
SystemError: opening file "/home/runner/work/LSINC1113/LSINC1113/Lectures/utils.jl":
No such file or directory
```

Show stack trace...

```
include("utils.jl")
```

Error message

```
UndefVarError: `load_biblio!` not defined in `Main.var"workspace#3"`
Suggestion: check for spelling errors or missing imports.
```

Show stack trace...

```
biblio = load_biblio!()
```

cite (generic function with 1 method)

```
cite(args...) = bibcite(biblio, args...)
```

refs (generic function with 1 method)

```
refs(keys) = bibrefs(biblio, keys)
```